FORGE & FABRIC INDUSTRIES, INC.

Selective Stage Pipeline, Enhanced Outsourcing, Kanban Overhaul & Hardcode Elimination

Complete Implementation Plan for Claude Code Execution
Document Version 2.0 — August 20, 2026

SCOPE: This plan covers four interrelated changes: (A) Customers select production services by *name* (not stage numbers) at the existing Step 2 'Production Request' in the intake wizard, with dynamic detail collection per service; (B) The factory can outsource any stage to external vendors with material-type-aware tracking, person-level dispatch/receive logging, and a mandatory QC gate on return; (C) The admin Kanban board is overhauled to show in-house vs outsource routing per order with blocking indicators; (D) All hardcoded fallbacks in the conversion pipeline are eliminated (size matrix, style names, colorways, PO numbers).

Table of Contents

1. Executive Summary & What Changes

This plan introduces two new requirements (**REQ-14** and **REQ-15**) plus a **Kanban board overhaul** and a **hardcode elimination sweep**, all designed to work together. Here is what each does:

REQ-14 — Selective Stage Pipeline: Right now customers see a 'Product Process Request' dropdown in Step 2 (StyleBlockEditor) with 4 rigid options: Full CMT, Sewing Only, Wash Only, Finish Only. This is replaced with a **multi-select service picker using friendly names** (e.g. 'Cutting & Bundling', 'Sewing Assembly', 'Washing & Laundry', 'Finishing & Effects') — customers never see 'stage 5' or 'stage 7'. When they select a service, the system auto-expands dynamic detail fields specific to that service (fabric details for cutting, wash recipe for washing, etc.). The system maps these friendly names to the correct stage numbers internally.

REQ-15 — Enhanced Outsourcing: Any stage can be outsourced to an external vendor. The outsource tracking is enhanced with material-type awareness (what goes out differs by stage), person-level dispatch/receive logging, quantity reconciliation, and a **mandatory QC inspection gate** on return that blocks stage advancement until passed. Customers see only progress — never whether work was done in-house or outsourced.

Kanban Overhaul: The admin Kanban board gets new card designs showing: which stages are in-house vs outsourced, outsource return QC status, blocking indicators, and the order's selected service scope. Cards for outsourced stages show vendor name and a 'Waiting for Return' or 'QC Pending' badge that blocks the advance button.

Hardcode Elimination: Fixes identified across the codebase where fallback values silently inject fake data into the production pipeline.

2. Hardcoded Issues Identified & Fixes

The following hardcoded fallbacks were found in the codebase. Each one silently injects fake data if the real data is missing, which means merchandisers and production staff can end up working with incorrect information without knowing it. **Every one of these must be fixed.**

File	Problem	Fix
ConversionModal.tsx (line ~extractedSizes)	Hardcoded fallback size matrix: { "28": 0, "30": 0, "32": 0, "34": 0, "36": 0, "38": 0 } when no size data found. These are denim-specific waist sizes that make no sense for T-shirts, kidswear, or knits.	Replace with empty object {}. If no size data exists, show a warning banner in the modal: 'No size matrix found in submission. Please enter sizes manually.' Force the merchandiser to input sizes rather than silently defaulting to wrong ones.
ConversionModal.tsx (styleName)	Fallback: 'STYLE-PROD' when no style name found. This generic string gets saved to the work order.	Remove fallback. If style_name/style_number is missing, show validation error: 'Style name is required.' Block conversion until provided.
ConversionModal.tsx (colorway)	Fallback: 'Standard Colorway' — a meaningless placeholder that gets saved as real data.	Remove fallback. Make colorway required with validation. If not provided by the customer, merchandiser must explicitly enter it.
ConversionModal.tsx (washType)	Fallback: 'Standard Finish' — another meaningless default.	Remove fallback. Make wash type required. If the order doesn't need washing (not in selected stages), set to 'N/A — Not Selected' automatically.
ConversionModal.tsx (poNumber)	Auto-generates PO numbers from reference codes or random numbers: PO-2026-XXXX. These are not real PO numbers from the customer.	Pre-fill from submission.existing_order_reference if available. Otherwise leave empty with a required field indicator. The merchandiser must enter or confirm the real PO number.
ConversionModal.tsx (woNumber)	Auto-generates WO numbers from reference codes: WO-2026-XXXX.	Auto-generation is acceptable for WO numbers (factory-internal), but use a proper sequential generator: query max existing WO number and increment, not random. Format: WO-{YYYY}-{sequential_5_digit}.
ConversionModal.tsx (dueDate)	Fallback: 45 days from today. This arbitrary default may not match any real delivery commitment.	Pre-fill from submission data if available (planned_ship_date, due_date). Otherwise leave empty with required validation. Use REQ-09 capacity calculator to suggest a date rather than defaulting to 45 days.
StyleBlockEditor.tsx (starting_stage)	Service scope dropdown maps to only 4 starting stages (1, 4, 7, 10). Does not support selective per-service stages.	Replace with the new multi-select service picker (REQ-14). Map to proper selected_stages array instead of a single starting_stage integer.
ConversionModal.tsx (startingStage)	Maps service_scope strings ('wash_only' → 9, 'sew_only' → 6, 'finish_only' → 12, default → 1). Only supports 4 routing paths.	Replace with reading selected_stages array from the submission's style block. Set order.current_stage to the first element of selected_stages. Remove the 4-way switch entirely.

3. REQ-14: Selective Stage Pipeline

3A. How Service Selection Works (Customer Sees Names, Not Numbers)

Customers should **never** see 'Stage 1', 'Stage 5', or 'Stage 9'. They see **service names** that describe what the factory will do for them. The system maps these to the correct internal stage numbers.

Service Name (Customer Sees)	Description	Internal Stages Mapped	Auto-Included Stages
Fabric Receiving & Inspection	We receive your fabric, inspect for defects, and log into inventory	1, 2, 3	Auto-included if any production service is selected and customer supplies raw fabric
Pre-Production Planning	We plan cutting layouts, line allocation, and scheduling	4	Auto-included if Cutting is selected
Cutting & Bundling	We cut your fabric into panels by size/color and bundle them for assembly	5, 6	Requires fabric receiving (1-3) unless customer supplies pre-cut panels
Sewing Assembly	We stitch cut panels into finished garments on industrial sewing lines	7	Requires cutting (5,6) unless customer supplies cut panels
Pre-Wash Quality Check	We inspect stitched garments before any washing or finishing	8	Auto-included if Sewing is selected
Washing & Laundry	Industrial washing, enzyme treatment, stonewash, softener, and drying	9	Optional — not all garments need washing
Finishing & Effects	Laser fading, ozone lightening, 3D creases, distressing, spray treatments	10	Optional — only for garments needing special effects
Final Quality Inspection	Comprehensive AQL inspection of the finished garment against the tech pack	11	Auto-included if any production service is selected
Pressing, Tagging & Packing	Steam press, hangtags, care labels, brand labels, carton packing	12	Auto-included if any production service is selected
Dispatch & Delivery	Final audit, shipping manifest, driver proof-of-delivery	13	Always included

KEY DESIGN DECISION: The customer selects from the 6 production services (Cutting, Sewing, Pre-Wash QC, Washing, Finishing, Pressing/Packing). The support stages (Receiving, Planning, Final QC, Dispatch) are **auto-included based on dependency rules** and shown as greyed-out 'included automatically' chips. This prevents the customer from accidentally skipping critical steps.

3B. Where It Lives: Step 2 in StyleBlockEditor.tsx

The service selection lives in **Step 2 (Order Details)** of the intake wizard, specifically inside each `StyleBlockEditor.tsx` component. This is where the current 'Product Process Request' dropdown already sits. We replace that dropdown with the new multi-select service picker.

Current code (to be replaced):

```
<select value={block.starting_stage || 1}>
  <option value={1}>Full CMT (Stage 1: Fabric Inspection & Cutting)</option>
  <option value={4}>Sewing Assembly Only (Stage 4: Panels Supplied)</option>
  <option value={7}>Wash & Laundry Processing Only (Stage 7: Garments Sewn)</option>
  <option value={10}>Finishing, Tagging & Packing Only (Stage 10)</option>
</select>
```

New replacement: ServiceScopeSelector component

- New file: `src/components/apply-portal/ServiceScopeSelector.tsx`
- Renders checkbox cards for each service group (not a dropdown)
- Each card shows: service name, short description, and an icon
- Selecting a card toggles it and auto-includes dependent stages
- A small pipeline preview strip at the bottom shows: 'Your order will pass through: Receiving → Cutting → Sewing → QC → Packing → Dispatch'
- Stores result as `block.selected_services: string[]` (service IDs) and `block.selected_stages: number[]` (internal stage numbers)

Preset shortcuts (shown above the service cards for common scenarios):

- **Full CMT** — selects all services (equivalent to all 13 stages)
- **Sew + Wash + Pack** — selects Sewing, Washing, Pressing/Packing (stages 7-13)
- **Wash + Finish Only** — selects Washing, Finishing, Pressing/Packing (stages 9-13)
- **Custom** — lets customer pick individual services

3C. Dynamic Detail Collection Per Selected Service

When a customer selects a service, the form dynamically expands to show fields specific to that service. This replaces the one-size-fits-all form that currently asks the same questions regardless of service scope.

Service Selected	Additional Fields Shown	Stored In
Cutting & Bundling	Fabric type (woven/knit/other), fabric weight (GSM/oz), estimated yardage, marker notes, any special cutting instructions	style_block.cutting_details (new JSON field)
Sewing Assembly	Thread color specs, stitch type preferences (single needle, double needle, chain stitch), label placement notes	style_block.sewing_details (new JSON field)

Service Selected	Additional Fields Shown	Stored In
Washing & Laundry	Wash recipe/type (enzyme, stonewash, bleach, silicone, garment dye, etc.), target color/shade, shrinkage tolerance, hand-feel target	style_block.wash_details (new JSON field — replaces the single wash_type text field)
Finishing & Effects	Laser pattern file reference, ozone level, 3D crease pattern, spray details, distressing level (light/medium/heavy)	style_block.finishing_details (new JSON field)
Pressing, Tagging & Packing	Hangtag specs, care label text, folding method (flat fold / hanger), poly bag required (Y/N), carton specs	style_block.packing_details (new JSON field)
Fabric Receiving (auto)	Number of fabric rolls expected, supplier name, expected delivery date to factory, inspection level (standard/premium)	style_block.receiving_details (new JSON field)

IMPORTANT: These detail fields are *optional* at intake — customers can submit without filling all of them, and the merchandiser fills in the blanks during the submission review process. The fields are shown to encourage the customer to provide more info up front, reducing back-and-forth. All of these details flow through to the submission inbox and the ConversionModal without any hardcoded fallbacks.

3D. Database Schema Changes

Table 1: orders

- New column: `selected_stages int[] DEFAULT '{1,2,3,4,5,6,7,8,9,10,11,12,13}'`
- Stores the internal stage numbers. Default = all 13 (backward-compatible with existing orders)
- The first element becomes `current_stage` on creation

Table 2: work_orders

- New column: `selected_stages int[] DEFAULT '{1,2,3,4,5,6,7,8,9,10,11,12,13}'`
- Inherited from the parent order at WO creation time

Table 3: apply_submissions

- New column: `requested_stages int[]` (nullable for legacy submissions)
- Populated from the style block's selected_stages during submission

No changes to the StyleBlockItem type in ApplyWizardContext.tsx — it already has `service_scope` and `starting_stage`. We add `selected_stages: number[]` and the new per-service detail JSON fields.

New helper function: get_next_selected_stage()

```
CREATE OR REPLACE FUNCTION get_next_selected_stage(p_order_id text, p_current int)
RETURNS int AS $$
DECLARE v_stages int[]; v_idx int;
BEGIN
  SELECT selected_stages INTO v_stages FROM orders WHERE order_id = p_order_id;
  IF v_stages IS NULL THEN RETURN p_current + 1; END IF;
  v_idx := array_position(v_stages, p_current);
  IF v_idx IS NULL OR v_idx >= array_length(v_stages, 1) THEN RETURN NULL; END IF;
  RETURN v_stages[v_idx + 1];
END; $$ LANGUAGE plpgsql STABLE;
```

3E. Submission Inbox & ConversionModal Data Flow

The submission inbox (`/submissions/$submissionId`) and the ConversionModal must display all the real data the customer submitted — no hardcoded fallbacks.

Submission detail page changes:

- Show 'Requested Services' badge strip: e.g. 'Cutting • Sewing • Washing • Packing' (derived from requested_stages)
- Show per-service details the customer provided (cutting_details, wash_details, etc.) in collapsible sections
- If the customer left a detail field empty, show it as 'Not provided — merchandiser to specify' in amber text, not a hardcoded default
- The size matrix displays exactly what the customer entered — if empty, show 'No sizes specified' warning, not {28:0, 30:0, ...}

ConversionModal changes (the big cleanup):

- Read selected_stages from the submission's style block and use it to set `orders.selected_stages` and `orders.current_stage` (first element of the array)
- Remove ALL hardcoded fallbacks listed in Section 2 — replace with required field validation
- Pre-fill fields from submission data where available; leave empty with required markers where not
- WO number auto-generation uses sequential query, not random
- Show a 'Pipeline Preview' in the modal: 'This order will follow: Receiving → Cutting → ... → Dispatch' based on selected_stages
- All per-service details from the customer are shown in the review step and carried forward to the work order

3F. Customer Portal Scoping

- The progress bar in `apply.status.$referenceCode.tsx` shows only the stages the customer selected
- Stage labels use the customer-friendly service names from the table in 3A, not internal stage names
- Progress % = (completed selected stages / total selected stages) × 100
- No visibility into in-house vs outsource routing (REQ-15 privacy shield)
- The `get_submission_status_by_reference()` RPC is extended to return `requested_stages`

3G. Stage-Gate Enforcement Updates

- **DB trigger:** `enforce_order_stage_gates()` — skip gate checks for stages not in `selected_stages`
- **Frontend:** `checkStageAdvancement()` in dashboard.tsx — respect selected_stages
- **Kanban advance:** `handleKanbanAdvance()` calls `getNextSelectedStage()` instead of `currentStage + 1`
- **Order detail:** Advance button on `orders.$orderId.tsx` uses `getNextSelectedStage()`
- **Backfill:** All existing orders get `selected_stages = {1..13}` where NULL (backward compat)

4. REQ-15: Enhanced Outsourcing with QC Return Gate

4A. Database Schema Enhancements

Extend: stage_outsourcing_records (add columns via ALTER TABLE, do not recreate):

Column	Type	Purpose
material_type	text NOT NULL DEFAULT 'general'	What goes out: 'fabric_rolls', 'cut_panels', 'stitched_garments', 'washed_garments', 'finished_garments', 'packed_cartons'
material_description	text	Free-text: '5 rolls of 12oz selvedge denim, lot #FL-2026-0042'
dispatched_by_user_id	uuid REFERENCES profiles(id)	Staff member who dispatched
dispatched_by_name	text NOT NULL	Denormalized name for audit trail
received_by_user_id	uuid REFERENCES profiles(id)	Staff member who received return
received_by_name	text	Filled on return
quantity_short	int GENERATED ALWAYS AS (quantity_dispatched - quantity_received) STORED	Auto-calculated shortage
return_qc_status	text DEFAULT 'Pending'	CHECK: Pending, Passed, Failed, Rework — blocks advancement until Passed
return_qc_inspection_id	uuid	Links to outsource_return_qc row
return_qc_notes	text	Inspector notes
transport_method	text	'Factory Truck', 'Third-Party Courier', 'Customer Pickup'
vehicle_reference	text	License plate / tracking number

New table: outsource_return_qc

```
CREATE TABLE IF NOT EXISTS public.outsource_return_qc (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  outsource_record_id uuid NOT NULL REFERENCES stage_outsourcing_records(id),
  order_id text NOT NULL,
  stage_number int NOT NULL,
  inspector_id uuid REFERENCES profiles(id),
  inspector_name text NOT NULL,
  inspected_qty int NOT NULL,
  passed_qty int NOT NULL DEFAULT 0,
  failed_qty int NOT NULL DEFAULT 0,
  rework_qty int NOT NULL DEFAULT 0,
  defect_notes text,
```

```
  photos text[],
  result text NOT NULL DEFAULT 'Pending'
    CHECK (result IN ('Pending','Passed','Failed','Rework','Partial_Pass')),
  inspected_at timestamptz DEFAULT now(),
  created_at timestamptz DEFAULT now()
);
```

RLS: Both tables: `USING (is_internal_staff())` — customers never see outsource data.

4B. Material Type by Stage (What Goes Out)

Service (Customer Name)	Stage #	Material Sent Out	material_type Value	What Returns
Fabric Receiving	1-3	Raw fabric rolls + trims	fabric_rolls	Inspected/sorted fabric
Pre-Production Planning	4	N/A (admin stage)	general	Planning docs
Cutting & Bundling	5-6	Fabric rolls + marker files	fabric_rolls	Cut panels by size/color
Sewing Assembly	7	Bundled cut panels	cut_panels	Stitched garments (unfinished)
Pre-Wash QC	8	Stitched garments	stitched_garments	QC-passed garments
Washing & Laundry	9	Stitched garments	stitched_garments	Washed garments
Finishing & Effects	10	Washed garments	washed_garments	Finished garments
Final QC	11	Finished garments	finished_garments	QC-passed garments
Pressing / Tagging / Packing	12	Finished garments	finished_garments	Packed cartons
Dispatch	13	Packed cartons	packed_cartons	Shipped (N/A)

This mapping lives in `src/lib/outsourcing-constants.ts` (new file) as a `STAGE_MATERIAL_MAP` constant.

4C. Outsourcing Dispatch & Receive UI

Enhanced StageOutsourcingPanel.tsx on orders.$orderId.tsx:

Dispatch Mode (sending work out):

- Stage selector — dropdown showing service names from that order's `selected_stages` only
- Material type auto-fills from STAGE_MATERIAL_MAP, with manual override option
- Material description (free text for lot numbers, roll counts, panel counts)
- Vendor name, facility location, outsource PO number
- Quantity dispatched (validated against order total)
- Dispatched by — auto-filled with logged-in user, editable
- Transport method + vehicle reference
- Expected return date

Receive Mode (receiving work back):

- 'Log Return' button on each Dispatched record

• Quantity received (cannot exceed qty dispatched)
• Received by — auto-filled with logged-in user
• Shortage auto-calculated, shown as red badge if > 0
• Vendor status auto-updates: 'Returned_Complete' or 'Returned_Partial'
• **Immediately opens QC return inspection form** (see 4D)
• Stage CANNOT advance until return_qc_status = 'Passed'

4D. Mandatory QC Return Gate (Blocks Advancement)

This is the most critical safety mechanism. **Every time outsourced work returns, it must pass QC before the order can advance to the next stage.**

Flow:

- **1.** Staff logs a return (Receive Mode above) → system auto-creates `outsource_return_qc` row with result='Pending'
- **2.** QC inspector sees 'Outsource Return QC' pending items in `/qc` (new section)
- **3.** Inspector logs: inspected qty, passed/failed/rework breakdown, defect notes, photos
- **4.** If Passed → `return_qc_status` set to 'Passed' → order can advance
- **5.** If Failed/Rework → order BLOCKED at current stage → rework_logs row for COPQ tracking

Stage-gate integration (critical logic):

The `enforce_order_stage_gates()` trigger is extended: before allowing advancement from any stage, check if that stage has ANY outsource records. If yes, ALL must have `return_qc_status = 'Passed'` (or 'Partial_Pass' if the business accepts partial). Otherwise block with: *'Outsourced work for [Service Name] has not passed return QC inspection.'*

This applies regardless of whether the order is advancing to the next sequential stage or jumping to a later stage via selective pipeline. The rule: **you cannot leave a stage until all outsourced work for that stage has returned and passed QC.**

The same check is mirrored in the frontend's `checkStageAdvancement()` function and in the Kanban board's advance button (both call the same helper), providing immediate visual feedback before the DB trigger is even hit.

4E. Customer Transparency Shield

Customer SEES: Progress against their selected services ('Cutting: Complete, Sewing: In Progress')

Customer does NOT see: Vendor names, outsource PO numbers, return QC details, dispatch/receive person names, shortage counts, cost data, or even the word 'outsource' anywhere.

- RLS on `stage_outsourcing_records`: `USING (is_internal_staff())`
- RLS on `outsource_return_qc`: same
- Customer portal components never import outsourcing-related code

5. Kanban Board Overhaul

5A. Current Problems

- Cards show minimal info: Order ID, Customer, PO#, Qty, Status
- No visibility into whether a stage is in-house or outsourced
- No indication of outsource return QC status or blocking conditions
- Advance button doesn't check outsource QC gate
- No way to see which services/stages were selected for each order
- Brand filter exists but cards lack service scope context

5B. New Kanban Card Design

Each Kanban card for an order is enhanced with:

Card Header:

- Order ID + Customer name (existing)
- Service scope badge strip: e.g. 'CUT • SEW • WASH • PACK' in small chips
- If order has Rush priority, show the existing amber RUSH badge

Card Body (new sections):

- **Current Stage**: 'Stage 7: Sewing Assembly' with the customer-friendly name
- **Routing indicator**: either a green 'IN-HOUSE' chip or an amber 'OUTSOURCED → [Vendor Name]' chip
- If outsourced and dispatched: show 'Dispatched [date] • Expected return [date]'
- If outsourced and returned: show 'Returned [date] • QC: Pending/Passed/Failed' with color coding
- **Shortage badge**: if quantity_short > 0 after return, show red 'SHORT: -[n] pcs'
- **Progress mini-bar**: thin colored bar showing completed stages / total selected stages

Card Footer (advance action):

- 'Advance Stage →' button: calls `getNextSelectedStage()`
- If outsource return QC is pending/failed, button shows a **locked state**: red lock icon + 'Blocked: Outsource QC Pending' tooltip
- If all gates pass, button is green and clickable
- If order is at final stage, show 'Dispatched & Completed' badge (existing)

5C. Outsource Blocking Indicators on Kanban

The Kanban advance button integrates with the outsource QC gate. The `checkStageAdvancement()` function is extended to include a new check:

```
// In checkStageAdvancement() — add before existing checks:
const outsourceRecords = outsource_data.filter(
  r => r.order_id === orderId && r.stage_number === currentStage
);
const pendingQC = outsourceRecords.filter(
  r => r.return_qc_status !== 'Passed' && r.return_qc_status !== 'Partial_Pass'
);
if (pendingQC.length > 0) {
  return {
    allowed: false,
    message: `Outsourced work for this stage has ${pendingQC.length} pending QC inspection(s). `
      + `Cannot advance until all return QC inspections pass.`
  };
}
```

6. Complete Data Flow Diagrams

Flow A: Customer Selects Partial Services at Intake

1. Customer opens /apply/new → Step 1: Company Info → Step 2: Order Details
2. In StyleBlockEditor, selects 'Sewing Assembly' + 'Washing & Laundry' + 'Pressing/Packing'
3. System auto-includes: Pre-Wash QC (8), Final QC (11), Dispatch (13)
4. Customer sees pipeline preview: 'Sewing → QC → Washing → Final QC → Packing → Dispatch'
5. Dynamic fields expand: sewing details + wash recipe + packing specs
6. Customer fills style matrix, sizes, uploads tech pack → submits
7. apply_submissions row created with requested_stages = {7,8,9,11,12,13}
8. Style blocks stored with all per-service detail JSON fields
9. Merchandiser opens /submissions/$id → sees 'Requested Services: Sewing, Washing, Packing'
10. Merchandiser reviews all details — no hardcoded fallbacks — fills in any blanks
11. Clicks 'Convert to Work Orders' → orders.selected_stages = {7,8,9,11,12,13}, current_stage = 7
12. Order appears in Kanban under 'Sewing & Finishing' column, card shows 'SEW • WASH • PACK' chips

Flow B: Factory Outsources a Stage

1. Order is at stage 7 (Sewing). Production manager decides to outsource sewing to 'ABC Sewing Co'
2. Opens /orders/$orderId → StageOutsourcingPanel → Dispatch Mode
3. Selects 'Sewing Assembly', material_type auto-fills as 'cut_panels'
4. Enters: vendor='ABC Sewing Co', qty=500, dispatched_by='Pat', transport='Factory Truck'
5. Record created: status='Dispatched', return_qc_status='Pending'
6. Kanban card for this order now shows: amber 'OUTSOURCED → ABC Sewing Co' chip
7. Advance button is LOCKED with tooltip 'Blocked: Outsource QC Pending'
8. 4 days later: 'Log Return' → qty_received=490, received_by='Joe', shortage=10
9. System auto-creates outsource_return_qc row with result='Pending'
10. QC inspector sees pending item in /qc → inspects → logs 480 pass, 10 rework
11. return_qc_status updates to 'Passed' → advance button unlocks
12. Kanban card shows green 'QC: Passed' chip, advance button is green
13. Customer portal shows: 'Sewing Assembly: Complete' — no mention of ABC Sewing Co

Flow C: Customer Supplies Pre-Cut Panels (Partial Pipeline)

Customer selects only: Sewing + Finishing + Packing. No cutting, no washing.
selected_stages = {7, 8, 10, 11, 12, 13}. Order starts at stage 7.
After stage 8 (Pre-Wash QC), next stage jumps to 10 (Finishing), skipping 9 (Washing).
Customer portal shows 6-stage progress bar. Dashboard pipeline greys out stages 1-6 and 9 for this order.

7. Screen-by-Screen Implementation Map

Screen / Route	REQ	What Changes
/apply/new (Step 2)	14	StyleBlockEditor's 'Product Process Request' dropdown replaced with ServiceScopeSelector multi-select cards. Dynamic detail fields expand per selected service. selected_stages computed from service selections.
/apply-intake (Step 2)	14	Same ServiceScopeSelector + merchandiser can override dependencies (e.g. skip auto-included QC).
/apply/status/$ref	14	Progress bar shows only requested_stages with customer-friendly names. No outsource info.
/submissions/$id	14 +Fix	Shows requested services as badge strip. Per-service detail sections. No hardcoded size matrix, style name, or colorway defaults. Missing fields shown as 'Not provided' in amber.
/submissions/$id (ConversionModal)	14 +Fix	Reads selected_stages from style blocks. All hardcoded fallbacks removed. Required validation on style, colorway, PO. WO number uses sequential generator. Pipeline preview shown.
/orders	14	Order list gets 'Services' column: 'CUT • SEW • WASH' chips or '13/13 Full CMT'.
/orders/$orderId	14 +15	Pipeline viz greys out non-selected stages. Enhanced StageOutsourcingPanel with Dispatch/Receive modes. Material type auto-fill. Person tracking. Outsource history table. Advance uses getNextSelectedStage().
/dashboard (Kanban)	14 +15	New card design: service scope chips, routing indicator (in-house/outsourced), vendor name, return QC status, shortage badge, progress mini-bar. Advance button checks outsource QC gate. getNextSelectedStage() for advancement.
/dashboard (Pipeline)	14	Stage drill-down shows 'N/A — Skipped' for orders that don't use that stage.
/cutting	15	If cutting outsourced, show 'Outsourced to [vendor]' badge, disable cutting form for that order.
/sewing	15	Same outsource badge pattern for sewing.
/wash	15	Same outsource badge pattern for washing.
/qc	15	New 'Outsource Return QC' tab: pending inspections, inspection form with pass/fail/rework + photos.
/reports	15	New 'Outsource Analytics': qty outsourced, return QC rates, shortage rates, vendor performance.

8. Migration SQL Specification

File: `supabase/migrations/20260820000000_selective_pipeline_and_enhanced_outsourcing.sql`

PATTERN: Every column guarded with ADD COLUMN IF NOT EXISTS, every policy with DROP POLICY IF EXISTS before CREATE POLICY. Migration is idempotent. Uses tenant_config (NOT tenant_branding).

Part 1: Add selected_stages to orders, work_orders, apply_submissions

Part 2: Create get_next_selected_stage() and get_prev_selected_stage() functions

Part 3: Add new columns to stage_outsourcing_records (material_type, dispatched_by, received_by, return_qc_status, etc.)

Part 4: Create outsource_return_qc table with RLS (is_internal_staff() only)

Part 5: Update enforce_order_stage_gates() to: (a) skip gates for non-selected stages, (b) check return_qc_status on outsourced stages

Part 6: Extend get_submission_status_by_reference() RPC to return requested_stages

Part 7: Performance indexes on new columns

Part 8: Backfill existing orders: selected_stages = '{1..13}' WHERE selected_stages IS NULL

9. Frontend File-by-File Changes

File	Action	Details
src/components/apply-portal/ServiceScopeSelector.tsx	NEW	Multi-select service cards with customer-friendly names, auto-dependency logic, pipeline preview strip. Replaces the 4-option dropdown.
src/lib/service-scope-constants.ts	NEW	SERVICE_GROUPS constant: maps service ID → { name, description, icon, stages[], autoIncludeIf[] }. PRESET_SCOPES: Full CMT, Sew+Wash+Pack, etc.
src/lib/outsourcing-constants.ts	NEW	STAGE_MATERIAL_MAP: stage → material_type + label. STAGE_FRIENDLY_NAMES: stage → customer name.
src/lib/validation/stageSelection.ts	NEW	Zod schema: int array, min 1 production stage, dependency rules as refinements.
src/hooks/useOutsourcing.ts	NEW	TanStack Query hooks: dispatch, receive, return QC, list by order.
src/contexts/ApplyWizardContext.tsx	MODIFY	Add to StyleBlockItem: selected_stages: number[], cutting_details, sewing_details, wash_details, finishing_details, packing_details, receiving_details (all optional JSON).
src/components/apply-portal/StyleBlockEditor.tsx	MODIFY	Replace 'Product Process Request' dropdown with ServiceScopeSelector. Add dynamic detail sections per selected service.
src/routes/apply.new.tsx	MODIFY	Wire requested_stages into apply_submissions insert payload.
src/routes/apply.status.$referenceCode.tsx	MODIFY	Filter progress by requested_stages. Use customer-friendly names.
src/components/merchandiser/ConversionModal.tsx	MAJOR FIX	Remove ALL hardcoded fallbacks (Section 2). Read selected_stages from style blocks. Required field validation. Sequential WO number. Pipeline preview.
src/routes/submissions.$submissionId.tsx	MODIFY	Show requested services badges. Per-service detail sections. Amber 'Not provided' for empty fields.
src/components/orders/StageOutsourcingPanel.tsx	MAJOR MODIFY	Split into Dispatch/Receive modes. Material type auto-fill. Person tracking. QC trigger. History table.
src/routes/orders.$orderId.tsx	MODIFY	Grey out non-selected stages. Advance uses getNextSelectedStage(). Show outsource routing status.
src/routes/dashboard.tsx	MODIFY	Enhanced Kanban cards: service chips, routing indicator, outsource QC status, shortage badge. Advance checks outsource QC. getNextSelectedStage().
src/routes/qc.tsx	MODIFY	New 'Outsource Return QC' tab with pending inspections, form, photos.
src/routes/cutting.tsx	MODIFY	Outsource badge if cutting outsourced.

File	Action	Details
src/routes/sewing.tsx	MODIFY	Outsource badge if sewing outsourced.
src/routes/wash.tsx	MODIFY	Outsource badge if washing outsourced.
src/routes/reports.tsx	MODIFY	Outsource analytics section.
src/lib/utils.ts	MODIFY	Add getNextSelectedStage() client-side mirror of DB function.
src/hooks/useAppData.tsx	MODIFY	Add outsource_data fetch + realtime subscription. Extend checkStageAdvancement with outsource QC check.

10. Build Order (4 Phases)

Phase 1: Database + Constants + Hardcode Fixes

- Write and apply the migration SQL (Section 8)
- Create `service-scope-constants.ts`, `outsourcing-constants.ts`, `validation/stageSelection.ts`
- Fix all hardcoded fallbacks in ConversionModal.tsx (Section 2)
- Add `getNextSelectedStage()` to utils.ts
- Update ApplyWizardContext.tsx type definitions
- **Smoke test:** `npx tsc --noEmit` clean; migration applies successfully; existing orders backfilled

Phase 2: Selective Stage Pipeline (REQ-14 Frontend)

- Build ServiceScopeSelector.tsx
- Wire into StyleBlockEditor.tsx (replace dropdown)
- Add dynamic detail sections per service
- Update submission payload to include requested_stages + per-service details
- Update /submissions/$id detail view
- Update ConversionModal to read selected_stages and set on orders/work_orders
- Update orders.$orderId.tsx pipeline visualization
- Update customer portal progress bar
- Update dashboard.tsx checkStageAdvancement + handleKanbanAdvance
- **Smoke test:** Create order with partial services; verify pipeline shows correct stages; customer portal shows correct progress

Phase 3: Enhanced Outsourcing (REQ-15 Frontend)

- Create useOutsourcing.ts hooks
- Rebuild StageOutsourcingPanel.tsx with Dispatch/Receive modes
- Build Outsource Return QC section in qc.tsx
- Add outsource badges to cutting.tsx, sewing.tsx, wash.tsx
- Extend checkStageAdvancement with outsource QC gate
- **Smoke test:** Dispatch → receive → QC blocks → QC passes → advance works

Phase 4: Kanban Overhaul + Analytics + Polish

- Redesign Kanban card layout in dashboard.tsx
- Add service scope chips, routing indicators, outsource status, shortage badges
- Add outsource analytics to reports.tsx
- End-to-end golden path test
- **Final:** `npx tsc --noEmit && npm run build` clean

11. Claude Code Prompt Sequence

Use sequentially. Wait for each to complete and verify before proceeding.

Prompt 1 (Phase 1):

```
"Read the implementation plan PDF in the project. Execute Phase 1: (1) Create migration
supabase/migrations/20260820000000_selective_pipeline_and_enhanced_outsourcing.sql implementing all
Parts 1-8 from Section 8 — follow project patterns (ADD COLUMN IF NOT EXISTS, DROP POLICY IF EXISTS,
idempotent). (2) Create src/lib/service-scope-constants.ts with SERVICE_GROUPS and PRESET_SCOPES from
Section 3A. (3) Create src/lib/outsourcing-constants.ts with STAGE_MATERIAL_MAP and
STAGE_FRIENDLY_NAMES from Section 4B. (4) Create src/lib/validation/stageSelection.ts Zod schema. (5)
Fix ALL hardcoded fallbacks in ConversionModal.tsx as specified in Section 2 — remove every fallback,
add required field validation, replace random WO number with sequential query. (6) Add
getNextSelectedStage() to src/lib/utils.ts. (7) Update ApplyWizardContext.tsx StyleBlockItem type to
add selected_stages and per-service detail fields. Run npx tsc --noEmit after."
```

Prompt 2 (Phase 2):

```
"Read the implementation plan PDF. Execute Phase 2: (1) Create
src/components/apply-portal/ServiceScopeSelector.tsx — multi-select service cards with friendly names
from service-scope-constants.ts, auto-dependency toggles, preset shortcuts (Full CMT / Sew+Wash+Pack /
Wash+Finish / Custom), pipeline preview strip. (2) Replace the Product Process Request dropdown in
StyleBlockEditor.tsx with ServiceScopeSelector. Add dynamic detail collection fields per selected
service (cutting_details, sewing_details, wash_details, finishing_details, packing_details,
receiving_details). (3) Update useApplySubmission.ts to include requested_stages and all per-service
details in the submission payload. (4) Update /submissions/$submissionId to show requested services
badges, per-service detail sections, and 'Not provided' amber warnings for empty fields — no hardcoded
defaults. (5) Update ConversionModal to read selected_stages from style blocks, set
orders.selected_stages and current_stage. Show pipeline preview. (6) Update orders.$orderId.tsx to grey
out non-selected stages and use getNextSelectedStage() for advancement. (7) Update
apply.status.$referenceCode.tsx to show only requested_stages with customer-friendly names. (8) Update
dashboard.tsx checkStageAdvancement and handleKanbanAdvance to use getNextSelectedStage(). Run npx tsc
--noEmit and npm run build."
```

Prompt 3 (Phase 3):

```
"Read the implementation plan PDF. Execute Phase 3: (1) Create src/hooks/useOutsourcing.ts with
TanStack Query hooks for outsource dispatch, receive, return QC, and list-by-order. (2) Rebuild
StageOutsourcingPanel.tsx with Dispatch mode (material type auto-fill from STAGE_MATERIAL_MAP, person
tracking, transport fields, qty validation) and Receive mode (qty received, shortage calc, auto-create
outsource_return_qc row). (3) Build 'Outsource Return QC' tab in qc.tsx: pending inspections list,
inspection form with pass/fail/rework + photo upload + defect notes, result propagation to
stage_outsourcing_records.return_qc_status. (4) Add outsource badges to cutting.tsx, sewing.tsx,
wash.tsx — if order has outsource record for that stage, show 'Outsourced to [vendor]' and disable the
normal form. (5) Extend checkStageAdvancement in useAppData.tsx: before allowing advance, check all
outsource records for current stage have return_qc_status Passed. (6) Add outsource_data fetch +
realtime subscription to useAppData.tsx. Ensure customer role cannot see any outsource data (RLS +
frontend). Run npx tsc --noEmit and npm run build."
```

Prompt 4 (Phase 4):

```
"Read the implementation plan PDF. Execute Phase 4: (1) Redesign Kanban cards in dashboard.tsx per
Section 5B: service scope chip strip, current stage with friendly name, routing indicator (green
IN-HOUSE / amber OUTSOURCED → vendor), outsource return QC status badge, shortage badge, progress
mini-bar. (2) Kanban advance button: locked with red indicator when outsource QC pending, green when
```

ready, with tooltip explaining blocked reason. (3) Add Outsource Analytics section to reports.tsx: total outsourced qty, return QC pass/fail rates, shortage rates, vendor performance ranked table. (4) Full end-to-end test: create order via /apply/new selecting only Cutting+Sewing+Packing services, outsource cutting to a vendor, log return with shortage, perform return QC, advance through remaining stages, verify customer portal shows only selected services with no outsource visibility, verify Kanban reflects all states correctly. Run npx tsc --noEmit and npm run build."

12. QA / Smoke Test Checklist

REQ-14: Selective Stage Pipeline

- [ ] /apply/new Step 2 shows ServiceScopeSelector with friendly service names (no stage numbers)
- [ ] Selecting 'Sewing Assembly' auto-includes 'Pre-Wash QC' and 'Final QC' and 'Packing' and 'Dispatch'
- [ ] Preset 'Full CMT' selects all services; preset 'Wash + Finish Only' selects correct subset
- [ ] Pipeline preview strip shows connected service names in order
- [ ] Selecting 'Washing' expands wash recipe fields; deselecting collapses them
- [ ] Submission payload includes requested_stages array + all per-service detail JSON
- [ ] /submissions/$id shows service badges and per-service details with 'Not provided' for empties
- [ ] ConversionModal has NO hardcoded fallbacks — empty size matrix shows warning, missing style blocks
- [ ] Converting with selected_stages={7,8,9,11,12,13} creates order with current_stage=7
- [ ] Order detail page greys out stages 1-6 and 10
- [ ] 'Advance' from stage 9 jumps to 11 (skipping 10)
- [ ] Customer portal shows 6-stage progress with friendly names
- [ ] Existing orders (before this feature) default to all 13 stages

REQ-15: Enhanced Outsourcing

- [ ] Dispatch form auto-fills material type from STAGE_MATERIAL_MAP
- [ ] Dispatched_by auto-fills with current user
- [ ] 'Log Return' validates qty ≤ qty dispatched
- [ ] Shortage auto-calculates and shows red badge
- [ ] Return creates outsource_return_qc row with result='Pending'
- [ ] /qc shows pending return QC inspection
- [ ] Inspector can log pass/fail/rework with photos
- [ ] Stage advancement BLOCKED when return_qc_status ≠ 'Passed'
- [ ] After QC passes, advancement ALLOWED
- [ ] cutting/sewing/wash pages show outsource badge for outsourced orders
- [ ] Customer portal shows NO outsource info
- [ ] Customer role cannot query outsource tables (RLS)

Kanban Board

- [ ] Cards show service scope chips (CUT • SEW • WASH)
- [ ] Cards show routing: green IN-HOUSE or amber OUTSOURCED → vendor
- [ ] Cards show return QC status with color coding
- [ ] Advance button LOCKED with red icon when outsource QC pending
- [ ] Advance button uses getNextSelectedStage (skips non-selected stages)
- [ ] Shortage badge shows on cards with qty_short > 0

Hardcode Elimination

- [ ] ConversionModal: empty size matrix shows warning, not {28,30,32,34,36,38}
- [ ] ConversionModal: missing style name shows validation error, not 'STYLE-PROD'
- [ ] ConversionModal: missing colorway shows validation error, not 'Standard Colorway'
- [ ] ConversionModal: missing wash type shows validation error, not 'Standard Finish'
- [ ] ConversionModal: empty PO field requires input, not auto-generated PO-2026-XXXX
- [ ] ConversionModal: WO number uses sequential generator, not random
- [ ] ConversionModal: due date uses capacity calculator suggestion or requires input, not 45-day default

Integration (All Requirements Together)

- [ ] Create order with services [Cutting, Sewing, Packing], outsource cutting → full flow works
- [ ] Create order with services [Washing, Finishing], outsource washing → full flow works
- [ ] Order with all 13 stages, no outsourcing → behaves exactly as before
- [ ] Outsourced stage + failed return QC → blocks until resolved
- [ ] Multiple outsource records for same stage → all must pass QC
- [ ] `npx tsc --noEmit` → zero errors
- [ ] `npm run build` → zero errors
